

SILLON — Tutoriel de découverte

SQL progressif, Python et R avancés sur un jeu de données réel

À propos de ce tutoriel

Ce document accompagne le compte de démonstration `demo@sillon.local` créé par le paquet `sillon-tutoriel`. Il vous fait découvrir SILLON à travers un vrai jeu de données ouvert, avec des exercices SQL en difficulté croissante, puis une initiation à l'analyse de données en Python et en R directement dans l'application.

Identifiants du compte de démonstration

Champ	Valeur
Adresse email	<code>demo@sillon.local</code>
Mot de passe	<code>demo</code>
Profil	Agent

Ce mot de passe est volontairement fixe et documenté, pour faciliter une session de démonstration ou de formation. Le paquet `sillon-tutoriel` ne doit **jamais** être installé sur un serveur de production — voir le guide d'installation administrateur, §7.2.

Le jeu de données

Deux tables sont déjà importées dans votre base personnelle :

- **`communes_france`** — 34 868 communes de France (métropole et outre-mer), avec pour chacune : son département, sa région, sa population, sa superficie, sa densité, son altitude et ses coordonnées.
- **`regions_france`** — les 18 régions françaises et le code INSEE de leur chef-lieu (préfecture de région), pour pratiquer les jointures.

Source : jeu de données « Communes et villes de France », publié sur data.gouv.fr, lui-même construit à partir des données officielles de l'INSEE et de l'IGN. Diffusé sous **Licence Ouverte / Open Licence version 2.0** (Etalab).

Colonnes de `communes_france` :

Colonne	Type	Description
code_insee	Texte	Code INSEE de la commune
nom_standard	Texte	Nom de la commune
dep_code	Texte	Code du département
dep_nom	Texte	Nom du département
reg_code	Texte	Code de la région
reg_nom	Texte	Nom de la région
population	Entier	Population municipale
superficie_km2	Décimal	Superficie en km²
densite	Décimal	Densité en habitants/km²
altitude_moyenne	Entier	Altitude moyenne en mètres
latitude_centre / longitude_centre	Décimal	Coordonnées du centre de la commune

Colonnes de `regions_france` : `reg_code`, `reg_nom`, `chef_lieu_nom`, `chef_lieu_code_insee`.

Partie 1 — SQL progressif

Tous les exercices se travaillent dans l'onglet **Travaux**, en sélectionnant votre base personnelle. Les sept premiers exercices sont déjà présents dans votre **historique** (onglet Travaux) : vous pouvez les relire ou les réexécuter avant de passer aux suivants, qui restent à écrire vous-même.

Niveau 1 — Sélection et filtrage

1.1. Affichez toutes les communes du département du Rhône (code `69`).

```
SELECT * FROM communes_france WHERE dep_code = '69';
```

1.2. Affichez uniquement le nom et la population des communes de plus de 50 000 habitants.

```
SELECT nom_standard, population FROM communes_france WHERE population > 50000;
```

1.3. *À vous de jouer* : affichez les communes de moins de 100 habitants situées en Corse (code région `94`).

▼ Corrigé

```
SELECT nom_standard, dep_nom, population
FROM communes_france
WHERE reg_code = '94' AND population < 100;
```

Niveau 2 — Tri et limitation

2.1. Les 10 communes les plus peuplées de Bretagne (code région 53).

```
SELECT nom_standard, population
FROM communes_france
WHERE reg_code = '53'
ORDER BY population DESC
LIMIT 10;
```

2.2. *À vous de jouer* : les 5 communes les plus hautes en altitude de France (attention aux valeurs manquantes : altitude_moyenne n'est pas toujours renseignée).

▼ Corrigé

```
SELECT nom_standard, dep_nom, altitude_moyenne
FROM communes_france
WHERE altitude_moyenne IS NOT NULL
ORDER BY altitude_moyenne DESC
LIMIT 5;
```

Niveau 3 — Agrégations

3.1. Nombre de communes et population totale du département de la Gironde (33).

```
SELECT COUNT(*) AS nb_communes, SUM(population) AS population_totale
FROM communes_france
WHERE dep_code = '33';
```

3.2. Population totale et nombre de communes par département, triés par population décroissante.

```
SELECT dep_nom, COUNT(*) AS nb_communes, SUM(population) AS population_totale
FROM communes_france
GROUP BY dep_nom
ORDER BY population_totale DESC;
```

3.3. *À vous de jouer* : la densité moyenne par région, arrondie à une décimale.

▼ Corrigé

```
SELECT reg_nom, ROUND(AVG(densite)::numeric, 1) AS densite_moyenne
FROM communes_france
GROUP BY reg_nom
ORDER BY densite_moyenne DESC;
```

Niveau 4 — Filtrer sur un agrégat (HAVING)

4.1. Les régions dont la densité moyenne dépasse 200 habitants/km².

```
SELECT reg_nom, ROUND(AVG(densite)::numeric, 1) AS densite_moyenne
FROM communes_france
GROUP BY reg_nom
HAVING AVG(densite) > 200
ORDER BY densite_moyenne DESC;
```

4.2. *À vous de jouer* : les départements comptant plus de 500 communes.

▼ Corrigé

```
SELECT dep_nom, COUNT(*) AS nb_communes
FROM communes_france
GROUP BY dep_nom
HAVING COUNT(*) > 500
ORDER BY nb_communes DESC;
```

Niveau 5 — Sous-requêtes

5.1. Les communes plus peuplées que la moyenne nationale.

```
SELECT nom_standard, dep_nom, population
FROM communes_france
WHERE population > (SELECT AVG(population) FROM communes_france)
ORDER BY population DESC;
```

5.2. *À vous de jouer* : pour chaque région, la commune la plus peuplée (indice : une sous-requête corrélée, ou une fonction de fenêtrage — voir niveau 7).

▼ Corrigé (sous-requête corrélée)

```
SELECT c.nom_standard, c.reg_nom, c.population
FROM communes_france c
WHERE c.population = (
    SELECT MAX(c2.population) FROM communes_france c2 WHERE c2.reg_nom =
    c.reg_nom
)
ORDER BY c.population DESC;
```

Niveau 6 — Jointures

6.1. Pour chaque région, la population de son chef-lieu (jointure avec `regions_france`).

```
SELECT c.nom_standard AS commune, c.population AS population_chef_lieu,
r.reg_nom
FROM communes_france c
JOIN regions_france r ON c.code_insee = r.chef_lieu_code_insee
ORDER BY c.population DESC;
```

6.2. *À vous de jouer* : la part de la population régionale que représente le chef-lieu (combinez la requête ci-dessus avec une agrégation de population par région).

▼ Corrigé

```
WITH population_regions AS (
    SELECT reg_nom, SUM(population) AS population_totale
    FROM communes_france
    GROUP BY reg_nom
)
SELECT
    c.nom_standard AS chef_lieu,
    c.population AS population_chef_lieu,
    pr.population_totale,
    ROUND(100.0 * c.population / pr.population_totale, 1) AS part_pourcent
FROM communes_france c
JOIN regions_france r ON c.code_insee = r.chef_lieu_code_insee
JOIN population_regions pr ON pr.reg_nom = r.reg_nom
ORDER BY part_pourcent DESC;
```

Niveau 7 — Fonctions de fenêtrage (avancé)

7.1. Le classement de chaque commune par population, au sein de son département (`RANK()` `OVER`).

```
SELECT
    nom_standard, dep_nom, population,
    RANK() OVER (PARTITION BY dep_code ORDER BY population DESC) AS
    rang_departemental
FROM communes_france
ORDER BY dep_nom, rang_departemental;
```

7.2. À vous de jouer : ne garder que la commune la plus peuplée de chaque département (indice : enveloppez la requête précédente et filtrez sur `rang_departemental = 1`).

▼ Corrigé

```
SELECT * FROM (
    SELECT
        nom_standard, dep_nom, population,
        RANK() OVER (PARTITION BY dep_code ORDER BY population DESC) AS
        rang_departemental
    FROM communes_france
) classement
WHERE rang_departemental = 1
ORDER BY population DESC;
```

Pour aller plus loin : le délai maximal d'une requête est limité (§11) ; une requête plus longue peut être basculée en tâche de fond (bouton « Exécuter en tâche de fond »), avec notification par mail à la fin — utile si vous étendez ces exercices à des calculs plus lourds.

Partie 2 — Python avancé

L'onglet **Scripts** permet de déposer un fichier `.py` ou `.R`, exécuté dans un environnement isolé avec un accès direct à votre base (variable d'environnement `SILLON_DSN`) et un répertoire de sortie pour vos résultats (`SILLON_RESULTATS`). Consultez la liste des librairies disponibles avant de coder (affichée dans l'onglet, ou dans `librairies-scripts.json`) : `pandas`, `numpy`, `matplotlib`, `psycpg2`, `openpyxl`.

Le script d'exemple

Un script `analyse_demographie.py` a déjà été déposé et exécuté sur votre compte (résultat téléchargeable depuis l'onglet **Suivi**). Il :

1. charge la table `communes_france` dans un `DataFrame` `pandas` ;
2. trace un histogramme horizontal des 15 communes les plus peuplées ;
3. calcule la population totale par région et la trace en barres ;
4. trace la distribution (en échelle logarithmique) de la densité de population.

Son squelette, à réutiliser comme point de départ :

```
import os
import psycpg2
import pandas as pd
import matplotlib
matplotlib.use("Agg") # obligatoire : pas d'affichage interactif possible
import matplotlib.pyplot as plt

dsn = os.environ["SILLON_DSN"]
resultats = os.environ["SILLON_RESULTATS"]

connexion = psycpg2.connect(dsn)
communes = pd.read_sql("SELECT * FROM communes_france", connexion)
connexion.close()

# ... votre analyse ...

figure, axe = plt.subplots()
# ... votre graphique ...
figure.savefig(os.path.join(resultats, "mon_graphique.png"))
```

Exercices

2.1. Calculez la superficie totale de la France métropolitaine à partir de `communes_france` (exclure les départements d'outre-mer, codes 971 à 976), et écrivez le résultat dans un fichier texte déposé dans `SILLON_RESULTATS`.

2.2. Tracez un nuage de points (`matplotlib.pyplot.scatter`) population/superficie des communes d'un département de votre choix, avec une échelle logarithmique sur les deux axes (`ax.set_xscale("log")` / `ax.set_yscale("log")`).

2.3. Avancé : calculez, pour chaque région, le coefficient de corrélation entre altitude moyenne et densité (`DataFrame.corr()` après un `groupby`). Les régions de montagne sont-elles significativement moins denses ?

2.4. Avancé : exportez un fichier Excel (`pandas.DataFrame.to_excel`, via `openpyxl`) contenant un onglet par région, avec le classement des communes de la région par population.

Partie 3 — R avancé

Le script d'exemple

Un script `analyse_densite.R` a été déposé et exécuté de la même façon (résultat dans l'onglet **Suivi**). Il utilise `DBI` / `RPostgreSQL` pour la connexion, `dplyr` pour la manipulation de données et

`ggplot2` pour les graphiques. Librairies R disponibles : `tidyverse` (dont `dplyr`, `ggplot2`, `tidyr`, `readr`), `DBI`, `RPostgreSQL`.

Point d'attention : `RPostgreSQL::dbConnect()` n'accepte pas directement la chaîne de connexion `SILLON_DSN` (format `clé=valeur` de libpq) — il faut l'analyser vous-même :

```
library(DBI)
library(RPostgreSQL)
library(dplyr)
library(ggplot2)

analyser_dsn <- function(dsn) {
  valeurs <- list()
  for (paire in strsplit(trimws(dsn), "\\s+")[1]) {
    cle_valeur <- strsplit(paire, "=", fixed = TRUE)[1]
    valeurs[[cle_valeur[1]]] <- cle_valeur[2]
  }
  valeurs
}

parametres <- analyser_dsn(Sys.getenv("SILLON_DSN"))
resultats <- Sys.getenv("SILLON_RESULTATS")

connexion <- dbConnect(
  PostgreSQL(),
  host = parametres$host, port = as.integer(parametres$port),
  dbname = parametres$dbname, user = parametres$user, password =
parametres$password
)
communes <- dbGetQuery(connexion, "SELECT * FROM communes_france")
dbDisconnect(connexion)
```

Le script fourni calcule ensuite la densité moyenne des 15 départements les plus denses (graphique en barres horizontales) et trace la relation entre altitude et densité (nuage de points, échantillonné à 3000 communes pour la lisibilité).

Exercices

3.1. Avec `dplyr`, calculez le nombre de communes et la population totale par région (`group_by(reg_nom) %>% summarise(...)`), et exportez le résultat en CSV avec `write.csv()`.

3.2. Tracez, avec `ggplot2`, un histogramme (`geom_histogram`) de la population des communes d'une région de votre choix.

3.3. Avancé : à l'aide de `regions_france`, faites une jointure (`inner_join` de `dplyr`) entre les deux tables pour comparer, région par région, la population du chef-lieu à la population totale de la région — comme l'exercice SQL 6.2, mais en R.

3.4. Avancé : tracez une carte simplifiée de France (`geom_point` sur `longitude_centre / latitude_centre` , taille ou couleur du point proportionnelle à la population) pour visualiser la répartition de la population sur le territoire.

Pour continuer

- Les onglets **Bases**, **Import** et **Suivi** couvrent le reste des fonctionnalités de SILLON : import de vos propres fichiers CSV, partage de base avec un autre compte, suivi et annulation des traitements en cours.
- Le **Guide d'installation administrateur** (accessible depuis le bouton « À propos ») détaille la configuration du serveur, les quotas et les procédures de désinstallation.
- Ce compte de démonstration peut être réinitialisé à tout moment : `sudo apt-get purge sillon-tutoriel && sudo apt-get install sillon-tutoriel` (voir le guide d'installation administrateur, §7.2).